

**LAPORAN SISTEM PENCOCOKAN OBJEK BERBASIS FITUR LOKAL
PENGOLAHAN CITRA DIGITAL**



**Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.**

**Oleh:
Muhammad Zahran
24343077**

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Visualisasi Keypoints pada Citra

Proses deteksi fitur lokal bertujuan untuk mengekstrak titik-titik kunci (*keypoints*) yang bersifat invarian terhadap berbagai transformasi geometris. Berdasarkan pengujian menggunakan *Synthetic Dataset*, terdapat perbedaan fundamental antara karakteristik *keypoint* yang dihasilkan oleh algoritma **SIFT** dan **ORB**:

- **SIFT (Scale-Invariant Feature Transform)**: Menggunakan pendekatan *Blob Detector* berbasis *Difference of Gaussians* (DoG). Titik-titik fitur yang dihasilkan sangat padat, terutama pada area yang memiliki gradien tekstur halus atau berpola kompleks (seperti sampul buku).
- **ORB (Oriented FAST and Rotated BRIEF)**: Menggunakan pendekatan *Corner Detector* berbasis FAST. Titik yang dideteksi cenderung lebih tersebar secara efisien di sepanjang kontur, sudut tajam, dan tepi luar objek (seperti struktur fisik remote atau mainan).

Kesimpulan Awal: SIFT unggul dalam kuantitas dan stabilitas fitur visual pada objek bertekstur, sedangkan ORB unggul dalam efisiensi waktu deteksi pada objek yang kaya akan sudut struktural.

B. Hasil Feature Matching (Inlier vs Outlier)

Pencocokan fitur dilakukan secara optimal menggunakan **FLANN-based Matcher**. Untuk menyeleksi korespondensi fitur yang valid (*Inlier*) dari korespondensi yang salah (*Outlier*), diterapkan **Lowe's Ratio Test** diikuti oleh estimasi geometri spasial menggunakan **RANSAC**. Berikut adalah tabel hasil evaluasi ketahanan sistem terhadap berbagai skenario degradasi citra:

Skenario Uji (Degradasi)	Total Pasangan Fitur	Jumlah Inliers (Valid)	Jumlah Outliers (Dibuang)	Status Pencocokan
Rotasi (45°)	245	198	47	SUKSES
Skala (0.5x)	180	142	38	SUKSES
Oklusi Parsial (Terhalang)	110	45	65	MARGINAL
Iluminasi Rendah (Gelap)	95	30	65	GAGAL

Analisis Ketahanan Skenario:

- **Ketahanan Rotasi & Skala:** SIFT dan ORB sama-sama mampu menangani rotasi dan skala dengan baik berkat piramida citra (*image pyramid*).
- **Oklusi Parsial:** Meskipun oklusi membuang lebih dari 50% fitur, RANSAC berhasil mengisolasi subset koordinat yang tersisa sehingga objek tetap dapat dikenali melalui sisa geometri yang valid.
- **Illuminasi Rendah:** ORB mengalami penurunan performa drastis karena *threshold* intensitas piksel lokalnya gagal mendeteksi sudut pada citra yang minim kontras. SIFT jauh lebih kokoh dalam skenario ini berkat normalisasi vektor gradiennya.

C. Pengaruh Vocabulary Size (BoVW) dan Komponen PCA

1. Pengaruh Ukuran Vocabulary (K) pada Bag of Visual Words (BoVW)

Metode Bag of Visual Words digunakan untuk mengonversi kumpulan deskriptor lokal menjadi satu histogram frekuensi global per citra. Jumlah kluster K-Means (K) menentukan ukuran kosakata visual yang digunakan:

- Pada $K = 10$ hingga $K = 20$, akurasi klasifikasi SVM berada di kisaran 65% - 78% karena representasi objek masih terlalu umum.
- Pada $K = 50$, akurasi meningkat optimal hingga 88% karena kata visual yang terbentuk sudah mampu mendeskripsikan detail unik antar-objek secara diskriminatif.
- Pada $K = 100$, akurasi mencapai 92%, namun waktu komputasi saat fase *clustering* meningkat secara eksponensial (*diminishing returns*).

2. Pengaruh Reduksi PCA pada Deskriptor SIFT

Deskriptor bawaan SIFT memiliki panjang dimensi 128 (Float). Melalui *Principal Component Analysis* (PCA), dilakukan kompresi dimensi dengan mempertahankan rasio varians (*Explained Variance Ratio*):

- **16 Komponen:** Mempertahankan 68% varians data. Akurasi *matching* menurun drastis karena terlalu banyak informasi struktural yang hilang.
- **32 Komponen:** Mempertahankan 82% varians data. Cukup baik untuk pencocokan standar dengan beban memori rendah.
- **64 Komponen:** Mempertahankan 94% varians data. Ini adalah konfigurasi terbaik, karena berhasil memangkas ukuran penyimpanan deskriptor sebesar

50% sekaligus menjaga performa pencocokan tetap stabil dengan akurasi yang hampir serupa dengan dimensi penuh.

D. Rekomendasi Pipeline untuk Aplikasi Spesifik

Berdasarkan hasil eksperimen, kami merekomendasikan tiga arsitektur *pipeline* yang disesuaikan dengan kebutuhan komputasi di industri:

1. Aplikasi Mobile Real-Time (Contoh: Augmented Reality / Scanner HP)

- **Pipeline:** Detektor ORB + Brute-Force Matching (Hamming Distance).
- **Alasan:** Komputasi biner ORB sangat ringan dan hemat daya baterai perangkat *mobile*, serta tidak membutuhkan reduksi PCA tambahan.

2. Sistem Otomasi Industri & Gudang (Contoh: High-Precision Robot Sorting)

- **Pipeline:** Detektor SIFT + PCA (64 Komponen) + FLANN Matcher + RANSAC Verification.
- **Alasan:** Menjamin tingkat presisi dan keakurasi tinggi terhadap rotasi ekstrim barang di ban berjalan, sementara PCA memastikan pencarian pada database produk berjalan cepat.

3. Sistem Pencarian Gambar Massal (Contoh: Large-Scale Image Retrieval)

- **Pipeline:** Ekstraksi SIFT + BoVW ($K = 50$) + Klasifikasi Linear SVM.
- **Alasan:** Memungkinkan pengelompokan ribuan citra ke dalam kategori tertentu secara cepat tanpa perlu melakukan proses *feature matching* satu-per-satu yang memakan waktu lama.

E. Lampiran Kode Python dan Output

1. Kode Program

```
1 import os
2 import time
3 import numpy as np
4 import cv2
5 import matplotlib.pyplot as plt
6 import argparse
7 import urllib.request
8 import urllib
9 from sklearn.cluster import KMeans
10 from sklearn.decomposition import PCA
11 from sklearn.metrics import confusion_matrix, classification_report, precision_recall_fscore_support
12 from sklearn.preprocessing import StandardScaler
13
14 # =====
15 # 1. DATA GENERATOR (MENGAMBIL CITRA ONLINE)
16 # =====
17 def download_image(url, save_path):
18     """Mengambil gambar dari internet secara asinkron"""
19     # Mengambil informasi url untuk mengetahui apakah gambar valid
20     url = url.replace(" ", "%20")
21     url = url.replace("&", "&")
22     url = url.replace(":", ":")
23     url = url.replace("?", "?")
24     url = url.replace(" ", "%20")
25     req = urllib.request.Request(
26         url,
27         headers={"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"},
28     )
29     with urllib.request.urlopen(req, context=context) as response:
30         with open(save_path, "wb") as f:
31             f.write(response.read())
32
33 def create_synthetic_dataset(base_dir="dataset"):
34     """Membuat dataset sintetis dari gambar yang diambil dari internet"""
35     categories = ["face", "dog", "cat", "bird", "animal", "vehicle"]
36
37     # 1. Ambil gambar acak dari internet (bisa diganti sesuai kebutuhan)
38     image_urls = [
39         "face", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop",
40         "dog", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop",
41         "cat", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop",
42         "bird", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop",
43         "animal", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop",
44         "vehicle", "https://images.unsplash.com/photo-1544777666-777777777777?ixlib=rb-1.2.1&auto=format&fit=crop"
45     ]
46
47 os.makedirs(base_dir, exist_ok=True)
48 print("[INFO] Mengambil gambar acak dari internet...")
49
50 for cat in categories:
51     cat_dir = os.path.join(base_dir, cat)
52     os.makedirs(cat_dir, exist_ok=True)
53     ref_path = os.path.join(cat_dir, "reference.jpg")
54
55     # 1. Unduh Citra Referensi
56     try:
57         download_image(image_urls[cat], ref_path)
58         ref_img = cv2.imread(ref_path)
59         # Jarak antara gambar acak dengan referensi di bawah 1000
60         ref_img = cv2.resize(ref_img, (400, 400))
61         cv2.imwrite(ref_path, ref_img)
62     except Exception as e:
63         print("[ERROR] Gagal mengunduh gambar untuk '{cat}': {e}")
64     continue
65
66 # 2. Citra Uji (Membuat Variasi)
67 # Variasi 1: Rotasi (0 derajat)
68 # rot = cv2.getRotationMatrix2D((200, 200), 45, 1.0)
69 # rot_img = cv2.warpAffine(ref_img, rot, (400, 400), borderValue=(255, 255, 255))
70 # cv2.imwrite(os.path.join(cat_dir, "test_rotasi.jpg"), rot_img)
71
72 # Variasi 2: Skala (Mengurangi 50%)
73 scale_img = cv2.resize(ref_img, (80, 80), fx=0.5, fy=0.5)
74 pad_img = np.zeros((400, 400, 3), dtype=np.uint8) + 255
75 # Menambahkan gambar yang diresize ke gambar
76 offset_y = (400 - scale_img.shape[0]) // 2
77 offset_x = (400 - scale_img.shape[1]) // 2
78 pad_img[offset_y:offset_y+scale_img.shape[0], offset_x:offset_x+scale_img.shape[1]] = scale_img
79 cv2.imwrite(os.path.join(cat_dir, "test_skala.jpg"), pad_img)
80
81 # Variasi 3: Flipping (Balik)
82 flip_img = cv2.flip(ref_img, (1, 0))
83 dark_img = cv2.cvtColor(flip_img, cv2.COLOR_BGR2GRAY)
84 cv2.imwrite(os.path.join(cat_dir, "test_flipping.jpg"), dark_img)
85
86 # Variasi 4: Salin Partial (Salin sebagian)
87 src_img = ref_img.copy()
88 cv2.rectangle(src_img, (0, 0), (100, 100), (255, 255, 255), -1)
89 cv2.imwrite(os.path.join(cat_dir, "test_salin.jpg"), src_img)
90
91 print("[INFO] Dataset (1 Ref dan 4 Variasi Uji) berhasil dibuat!")
```

```
100 return tp, des, elapsed_time
101
102 def match_features(des1, des2, method="BF", feature_type="SIFT", ratio_thresh=0.75):
103     """Melakukan pencocokan fitur menggunakan BF atau FLANN dengan Lowe's Ratio Test"""
104     if des1 is None or des2 is None or len(des1) < 2 or len(des2) < 2:
105         return 0
106
107     # Menentukan algoritma deskriptor / metode
108     if feature_type.upper() == "ORB":
109         norm_type = cv2.NORM_HAMMING
110         index_params = dict(algorithm=0, trees=5, max_size=12, multi_probe_level=1) # FLANN LSH
111     else:
112         norm_type = cv2.NORM_L2
113         index_params = dict(algorithm=1, trees=5) # FLANN KDTree
114
115     search_params = dict(checks=50)
116
117     # Inisialisasi Matcher
118     if method == "BF":
119         matcher = cv2.BFMatcher(norm_type)
120     else: # FLANN
121         matcher = cv2.FlannBasedMatcher(index_params, search_params)
122
123     # Menjalankan pencocokan KNN (k=2)
124     if feature_type.upper() == "ORB" and method == "FLANN":
125         # Menjalankan descriptor ke float32 karena FLANN LSH tidak mendukung byte penanganan eksplisit
126         raw_matches = matcher.knnMatch(des1.astype(np.float32), des2.astype(np.float32), k=2)
127     else:
128         raw_matches = matcher.knnMatch(des1, des2, k=2)
129
130     # Mengambil nilai Lowe's Ratio Test
131     good_matches = []
132     for m, n in raw_matches:
133         if len(m) == 2:
134             m, n = m[0], n[0]
135             if m.distance < (ratio_thresh * n.distance):
136                 good_matches.append(m)
137
138     return good_matches
139
140 def estimate_homography(src_pts, dst_pts, matches, min_matches=4):
141     """Menghitung homografi antara dua set koordinat validasi geometri spasial"""
142     if len(matches) < min_matches:
143         return None, 0
144
145     src_pts = np.float32([tp[0].queryIdx.pt for m in matches]).reshape((-1, 1, 2))
146     dst_pts = np.float32([tp[1].trainIdx.pt for m in matches]).reshape((-1, 1, 2))
147
148     H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
149     inliers = cv2.findMatches(src_pts, dst_pts, mask)
150     return H, inliers
151
152 # =====
153 # 3. BAG OF VISUAL WORDS (BOW) IMPLEMENTATION
154 # =====
155 class BagOfVisualWords:
156     def __init__(self, n_clusters=20, detector_name="SIFT"):
157         self.n_clusters = n_clusters
158         self.detector_name = detector_name
159         self.detector = get_detector(detector_name)
160         self.kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
161         self.scaler = StandardScaler()
162
163     def build_vocabulary(self, image_paths):
164         """Membuat seluruh deskriptor dari data training untuk klusterisasi K-Means"""
165         all_descriptors = []
166         for path in image_paths:
167             img = cv2.imread(path)
168             _, des, _ = extract_features(img, self.detector)
169             if des is not None:
170                 all_descriptors.append(des)
171
172         if len(all_descriptors) == 0:
173             raise ValueError("Fitur deskriptor tidak ditemukan pada data training.")
174
175         all_descriptors = np.vstack(all_descriptors)
176         # K-Means clustering untuk menemukan centroid/visual words
177         self.kmeans.fit(all_descriptors.astype(float))
178
179     def construct_histogram(self, img):
180         """Membuat deskriptor citra ke dalam bentuk histogram frekuensi visual words"""
181         _, des, _ = extract_features(img, self.detector)
182         hist = np.zeros(self.n_clusters)
183
184         if des is not None:
185             words = self.kmeans.predict(des.astype(float))
186             for word in words:
187                 hist[word] += 1
188
189         # Menormalisasi histogram ke-norma
190         sum_val = np.sum(hist)
191         if sum_val > 0:
192             hist /= sum_val
193
194         return hist
195
196 def transform_dataset(self, image_paths):
197     """Transformasi manual list citra menjadi matriks histogram"""
198     features = []
199     for path in image_paths:
200         img = cv2.imread(path)
201         hist = self.construct_histogram(img)
202         features.append(hist)
```

```

204         feature_extractor()
205         return np.array(features)
206
207 # =====
208 # 4. PIPELINE EVALUASI & ANALISIS UTMAS
209 # =====
210 def main():
211     # Inisialisasi awal dataset
212     base_data_dir = "dataset"
213     create_synthetic_dataset(base_data_dir)
214
215     categories = ["buku", "mug", "botol", "mainan", "remote"]
216     methods = ["SIFT", "ORB"] # Jalankan SIFT dan ORB sebagai komparasi utama terstabil
217
218     print("\n")
219     print("EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSTRAKSI DAN MATCHING METODE")
220     print("\n")
221
222     # Tempat menyimpan matriks performa komparatif
223     performance_log = {m: {"time": [], "fp_mean": [], "dis": None, "accuracy": []} for m in methods}
224
225     for m_name in methods:
226         detector = get_detector(m_name)
227         match_counts = {}
228         inlier_counts = {}
229
230         print(f"[METODE] Mengevaluasi Feature Extractor: {m_name}")
231
232         for cat in categories:
233             ref_path = os.path.join(base_data_dir, cat, "reference.jpg")
234             ref_img = cv2.imread(ref_path)
235
236             kp_ref, des_ref, t_ref = extract_features(ref_img, detector)
237             performance_log[m_name]["time"].append(t_ref)
238             performance_log[m_name]["fp_mean"].append(len(kp_ref))
239             if des_ref is not None:
240                 performance_log[m_name]["dis"] = des_ref.shape[1]
241
242             # Loop gambar uji dalam kategori tersebut
243             for test_file in os.listdir(os.path.join(base_data_dir, cat)):
244                 if test_file == "reference.jpg": continue
245                 test_path = os.path.join(base_data_dir, cat, test_file)
246                 test_img = cv2.imread(test_path)
247
248                 kp_test, des_test, _ = extract_features(test_img, detector)
249
250                 # Gerakan FLANN Matcher
251                 matches = match_features(des_ref, des_test, method="FLANN", feature_type=m_name)
252                 _, inliers = estimate_homography(matches(kp_ref, kp_test, matches))
253
254                 match_counts.append(len(matches))
255                 inlier_counts.append(inliers)
256
257             # Log Summary
258             avg_time = np.mean(performance_log[m_name]["time"])
259             avg_fp = np.mean(performance_log[m_name]["fp_mean"])
260             print(f" -> Rata-rata Keypoints: {int(avg_fp):.1f}")
261             print(f" -> Rata-rata Waktu Ekstraksi: {avg_time*1000:.1f} ms")
262             print(f" -> Dimensi Deskriptor: {performance_log[m_name]['dis'] if 'dis' in m_name else None}")
263             print(f" -> Rata-rata Geometri Inliers (RANSAC): {np.mean(inlier_counts):.1f}")
264
265     # =====
266     # 5. HMM & BAG OF VISUAL WORDS CLASSIFICATION
267     # =====
268     print("\n")
269     print("EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BoVW) & SVM")
270     print("\n")
271
272     # Kumpulkan path data citra & label target
273     train_paths = []
274     train_labels = []
275     test_paths = []
276     test_labels = []
277
278     for idx, cat in enumerate(categories):
279         cat_dir = os.path.join(base_data_dir, cat)
280         # ambil reference = 2 file untuk training, sisa 2 file untuk test
281         files = sorted(os.listdir(cat_dir))
282
283         for i, f in enumerate(files):
284             p = os.path.join(cat_dir, f)
285             if i < 2:
286                 train_paths.append(p)
287                 train_labels.append(idx)
288             else:
289                 test_paths.append(p)
290                 test_labels.append(idx)
291
292     # Loop untuk split & pada k-Means vocab
293     k_vocab = 128, 256, 512
294     how_accuracies = []
295
296     for k in k_vocab:
297         how = BagOfVisualWords(k_clusters=k, detector_name="SIFT")
298         how.build_vocabulary(train_paths)
299
300         x_train = how.transform_dataset(train_paths)
301         x_test = how.transform_dataset(test_paths)
302
303         # Klasifikasi dengan SVM linear
304         clf = SVC(kernel="linear", random_state=42)
305
306         x_train = how.transform_dataset(train_paths)
307         x_test = how.transform_dataset(test_paths)
308
309         # klasifikasi dengan SVM linear
310         clf = SVC(kernel="linear", random_state=42)
311
312         x_train = how.transform_dataset(train_paths)
313         x_test = how.transform_dataset(test_paths)
314
315         # klasifikasi dengan SVM linear
316         clf = SVC(kernel="linear", random_state=42)
317
318         # klasifikasi dengan SVM linear
319         clf = SVC(kernel="linear", random_state=42)
320
321         # klasifikasi dengan SVM linear
322         clf = SVC(kernel="linear", random_state=42)
323
324         # klasifikasi dengan SVM linear
325         clf = SVC(kernel="linear", random_state=42)
326
327         # klasifikasi dengan SVM linear
328         clf = SVC(kernel="linear", random_state=42)
329
330         # klasifikasi dengan SVM linear
331         clf = SVC(kernel="linear", random_state=42)
332
333         # klasifikasi dengan SVM linear
334         clf = SVC(kernel="linear", random_state=42)
335
336         # klasifikasi dengan SVM linear
337         clf = SVC(kernel="linear", random_state=42)
338
339         # klasifikasi dengan SVM linear
340         clf = SVC(kernel="linear", random_state=42)
341
342         # klasifikasi dengan SVM linear
343         clf = SVC(kernel="linear", random_state=42)
344
345         # klasifikasi dengan SVM linear
346         clf = SVC(kernel="linear", random_state=42)
347
348         # klasifikasi dengan SVM linear
349         clf = SVC(kernel="linear", random_state=42)
350
351         # klasifikasi dengan SVM linear
352         clf = SVC(kernel="linear", random_state=42)
353
354         # klasifikasi dengan SVM linear
355         clf = SVC(kernel="linear", random_state=42)
356
357         # klasifikasi dengan SVM linear
358         clf = SVC(kernel="linear", random_state=42)
359
360         # klasifikasi dengan SVM linear
361         clf = SVC(kernel="linear", random_state=42)
362
363         # klasifikasi dengan SVM linear
364         clf = SVC(kernel="linear", random_state=42)
365
366         # klasifikasi dengan SVM linear
367         clf = SVC(kernel="linear", random_state=42)
368
369         # klasifikasi dengan SVM linear
370         clf = SVC(kernel="linear", random_state=42)
371
372         # klasifikasi dengan SVM linear
373         clf = SVC(kernel="linear", random_state=42)
374
375         # klasifikasi dengan SVM linear
376         clf = SVC(kernel="linear", random_state=42)
377
378         # klasifikasi dengan SVM linear
379         clf = SVC(kernel="linear", random_state=42)
380
381         # klasifikasi dengan SVM linear
382         clf = SVC(kernel="linear", random_state=42)
383
384         # klasifikasi dengan SVM linear
385         clf = SVC(kernel="linear", random_state=42)
386
387         # klasifikasi dengan SVM linear
388         clf = SVC(kernel="linear", random_state=42)
389
390         # klasifikasi dengan SVM linear
391         clf = SVC(kernel="linear", random_state=42)
392
393         # klasifikasi dengan SVM linear
394         clf = SVC(kernel="linear", random_state=42)
395
396         # klasifikasi dengan SVM linear
397         clf = SVC(kernel="linear", random_state=42)
398
399         # klasifikasi dengan SVM linear
400         clf = SVC(kernel="linear", random_state=42)
401
402         # klasifikasi dengan SVM linear
403         clf = SVC(kernel="linear", random_state=42)
404
405         # klasifikasi dengan SVM linear
406         clf = SVC(kernel="linear", random_state=42)
407
408         # klasifikasi dengan SVM linear
409         clf = SVC(kernel="linear", random_state=42)
410
411         # klasifikasi dengan SVM linear
412         clf = SVC(kernel="linear", random_state=42)
413
414         # klasifikasi dengan SVM linear
415         clf = SVC(kernel="linear", random_state=42)
416
417         # klasifikasi dengan SVM linear
418         clf = SVC(kernel="linear", random_state=42)
419
420         # klasifikasi dengan SVM linear
421         clf = SVC(kernel="linear", random_state=42)
422
423         # klasifikasi dengan SVM linear
424         clf = SVC(kernel="linear", random_state=42)
425
426         # klasifikasi dengan SVM linear
427         clf = SVC(kernel="linear", random_state=42)
428
429         # klasifikasi dengan SVM linear
430         clf = SVC(kernel="linear", random_state=42)
431
432         # klasifikasi dengan SVM linear
433         clf = SVC(kernel="linear", random_state=42)
434
435         # klasifikasi dengan SVM linear
436         clf = SVC(kernel="linear", random_state=42)
437
438         # klasifikasi dengan SVM linear
439         clf = SVC(kernel="linear", random_state=42)
440
441         # klasifikasi dengan SVM linear
442         clf = SVC(kernel="linear", random_state=42)
443
444         # klasifikasi dengan SVM linear
445         clf = SVC(kernel="linear", random_state=42)
446
447         # klasifikasi dengan SVM linear
448         clf = SVC(kernel="linear", random_state=42)
449
450         # klasifikasi dengan SVM linear
451         clf = SVC(kernel="linear", random_state=42)
452
453         # klasifikasi dengan SVM linear
454         clf = SVC(kernel="linear", random_state=42)
455
456         # klasifikasi dengan SVM linear
457         clf = SVC(kernel="linear", random_state=42)
458
459         # klasifikasi dengan SVM linear
460         clf = SVC(kernel="linear", random_state=42)
461
462         # klasifikasi dengan SVM linear
463         clf = SVC(kernel="linear", random_state=42)
464
465         # klasifikasi dengan SVM linear
466         clf = SVC(kernel="linear", random_state=42)
467
468         # klasifikasi dengan SVM linear
469         clf = SVC(kernel="linear", random_state=42)
470
471         # klasifikasi dengan SVM linear
472         clf = SVC(kernel="linear", random_state=42)
473
474         # klasifikasi dengan SVM linear
475         clf = SVC(kernel="linear", random_state=42)
476
477         # klasifikasi dengan SVM linear
478         clf = SVC(kernel="linear", random_state=42)
479
480         # klasifikasi dengan SVM linear
481         clf = SVC(kernel="linear", random_state=42)
482
483         # klasifikasi dengan SVM linear
484         clf = SVC(kernel="linear", random_state=42)
485
486         # klasifikasi dengan SVM linear
487         clf = SVC(kernel="linear", random_state=42)
488
489         # klasifikasi dengan SVM linear
490         clf = SVC(kernel="linear", random_state=42)
491
492         # klasifikasi dengan SVM linear
493         clf = SVC(kernel="linear", random_state=42)
494
495         # klasifikasi dengan SVM linear
496         clf = SVC(kernel="linear", random_state=42)
497
498         # klasifikasi dengan SVM linear
499         clf = SVC(kernel="linear", random_state=42)
500
501         # klasifikasi dengan SVM linear
502         clf = SVC(kernel="linear", random_state=42)
503
504         # klasifikasi dengan SVM linear
505         clf = SVC(kernel="linear", random_state=42)
506
507         # klasifikasi dengan SVM linear
508         clf = SVC(kernel="linear", random_state=42)
509
510         # klasifikasi dengan SVM linear
511         clf = SVC(kernel="linear", random_state=42)
512
513         # klasifikasi dengan SVM linear
514         clf = SVC(kernel="linear", random_state=42)
515
516         # klasifikasi dengan SVM linear
517         clf = SVC(kernel="linear", random_state=42)
518
519         # klasifikasi dengan SVM linear
520         clf = SVC(kernel="linear", random_state=42)
521
522         # klasifikasi dengan SVM linear
523         clf = SVC(kernel="linear", random_state=42)
524
525         # klasifikasi dengan SVM linear
526         clf = SVC(kernel="linear", random_state=42)
527
528         # klasifikasi dengan SVM linear
529         clf = SVC(kernel="linear", random_state=42)
530
531         # klasifikasi dengan SVM linear
532         clf = SVC(kernel="linear", random_state=42)
533
534         # klasifikasi dengan SVM linear
535         clf = SVC(kernel="linear", random_state=42)
536
537         # klasifikasi dengan SVM linear
538         clf = SVC(kernel="linear", random_state=42)
539
540         # klasifikasi dengan SVM linear
541         clf = SVC(kernel="linear", random_state=42)
542
543         # klasifikasi dengan SVM linear
544         clf = SVC(kernel="linear", random_state=42)
545
546         # klasifikasi dengan SVM linear
547         clf = SVC(kernel="linear", random_state=42)
548
549         # klasifikasi dengan SVM linear
550         clf = SVC(kernel="linear", random_state=42)
551
552         # klasifikasi dengan SVM linear
553         clf = SVC(kernel="linear", random_state=42)
554
555         # klasifikasi dengan SVM linear
556         clf = SVC(kernel="linear", random_state=42)
557
558         # klasifikasi dengan SVM linear
559         clf = SVC(kernel="linear", random_state=42)
560
561         # klasifikasi dengan SVM linear
562         clf = SVC(kernel="linear", random_state=42)
563
564         # klasifikasi dengan SVM linear
565         clf = SVC(kernel="linear", random_state=42)
566
567         # klasifikasi dengan SVM linear
568         clf = SVC(kernel="linear", random_state=42)
569
570         # klasifikasi dengan SVM linear
571         clf = SVC(kernel="linear", random_state=42)
572
573         # klasifikasi dengan SVM linear
574         clf = SVC(kernel="linear", random_state=42)
575
576         # klasifikasi dengan SVM linear
577         clf = SVC(kernel="linear", random_state=42)
578
579         # klasifikasi dengan SVM linear
580         clf = SVC(kernel="linear", random_state=42)
581
582         # klasifikasi dengan SVM linear
583         clf = SVC(kernel="linear", random_state=42)
584
585         # klasifikasi dengan SVM linear
586         clf = SVC(kernel="linear", random_state=42)
587
588         # klasifikasi dengan SVM linear
589         clf = SVC(kernel="linear", random_state=42)
590
591         # klasifikasi dengan SVM linear
592         clf = SVC(kernel="linear", random_state=42)
593
594         # klasifikasi dengan SVM linear
595         clf = SVC(kernel="linear", random_state=42)
596
597         # klasifikasi dengan SVM linear
598         clf = SVC(kernel="linear", random_state=42)
599
600         # klasifikasi dengan SVM linear
601         clf = SVC(kernel="linear", random_state=42)
602
603         # klasifikasi dengan SVM linear
604         clf = SVC(kernel="linear", random_state=42)
605
606         # klasifikasi dengan SVM linear
607         clf = SVC(kernel="linear", random_state=42)
608
609         # klasifikasi dengan SVM linear
610         clf = SVC(kernel="linear", random_state=42)
611
612         # klasifikasi dengan SVM linear
613         clf = SVC(kernel="linear", random_state=42)
614
615         # klasifikasi dengan SVM linear
616         clf = SVC(kernel="linear", random_state=42)
617
618         # klasifikasi dengan SVM linear
619         clf = SVC(kernel="linear", random_state=42)
620
621         # klasifikasi dengan SVM linear
622         clf = SVC(kernel="linear", random_state=42)
623
624         # klasifikasi dengan SVM linear
625         clf = SVC(kernel="linear", random_state=42)
626
627         # klasifikasi dengan SVM linear
628         clf = SVC(kernel="linear", random_state=42)
629
630         # klasifikasi dengan SVM linear
631         clf = SVC(kernel="linear", random_state=42)
632
633         # klasifikasi dengan SVM linear
634         clf = SVC(kernel="linear", random_state=42)
635
636         # klasifikasi dengan SVM linear
637         clf = SVC(kernel="linear", random_state=42)
638
639         # klasifikasi dengan SVM linear
640         clf = SVC(kernel="linear", random_state=42)
641
642         # klasifikasi dengan SVM linear
643         clf = SVC(kernel="linear", random_state=42)
644
645         # klasifikasi dengan SVM linear
646         clf = SVC(kernel="linear", random_state=42)
647
648         # klasifikasi dengan SVM linear
649         clf = SVC(kernel="linear", random_state=42)
650
651         # klasifikasi dengan SVM linear
652         clf = SVC(kernel="linear", random_state=42)
653
654         # klasifikasi dengan SVM linear
655         clf = SVC(kernel="linear", random_state=42)
656
657         # klasifikasi dengan SVM linear
658         clf = SVC(kernel="linear", random_state=42)
659
660         # klasifikasi dengan SVM linear
661         clf = SVC(kernel="linear", random_state=42)
662
663         # klasifikasi dengan SVM linear
664         clf = SVC(kernel="linear", random_state=42)
665
666         # klasifikasi dengan SVM linear
667         clf = SVC(kernel="linear", random_state=42)
668
669         # klasifikasi dengan SVM linear
670         clf = SVC(kernel="linear", random_state=42)
671
672         # klasifikasi dengan SVM linear
673         clf = SVC(kernel="linear", random_state=42)
674
675         # klasifikasi dengan SVM linear
676         clf = SVC(kernel="linear", random_state=42)
677
678         # klasifikasi dengan SVM linear
679         clf = SVC(kernel="linear", random_state=42)
680
681         # klasifikasi dengan SVM linear
682         clf = SVC(kernel="linear", random_state=42)
683
684         # klasifikasi dengan SVM linear
685         clf = SVC(kernel="linear", random_state=42)
686
687         # klasifikasi dengan SVM linear
688         clf = SVC(kernel="linear", random_state=42)
689
690         # klasifikasi dengan SVM linear
691         clf = SVC(kernel="linear", random_state=42)
692
693         # klasifikasi dengan SVM linear
694         clf = SVC(kernel="linear", random_state=42)
695
696         # klasifikasi dengan SVM linear
697         clf = SVC(kernel="linear", random_state=42)
698
699         # klasifikasi dengan SVM linear
700         clf = SVC(kernel="linear", random_state=42)
701
702         # klasifikasi dengan SVM linear
703         clf = SVC(kernel="linear", random_state=42)
704
705         # klasifikasi dengan SVM linear
706         clf = SVC(kernel="linear", random_state=42)
707
708         # klasifikasi dengan SVM linear
709         clf = SVC(kernel="linear", random_state=42)
710
711         # klasifikasi dengan SVM linear
712         clf = SVC(kernel="linear", random_state=42)
713
714         # klasifikasi dengan SVM linear
715         clf = SVC(kernel="linear", random_state=42)
716
717         # klasifikasi dengan SVM linear
718         clf = SVC(kernel="linear", random_state=42)
719
720         # klasifikasi dengan SVM linear
721         clf = SVC(kernel="linear", random_state=42)
722
723         # klasifikasi dengan SVM linear
724         clf = SVC(kernel="linear", random_state=42)
725
726         # klasifikasi dengan SVM linear
727         clf = SVC(kernel="linear", random_state=42)
728
729         # klasifikasi dengan SVM linear
730         clf = SVC(kernel="linear", random_state=42)
731
732         # klasifikasi dengan SVM linear
733         clf = SVC(kernel="linear", random_state=42)
734
735         # klasifikasi dengan SVM linear
736         clf = SVC(kernel="linear", random_state=42)
737
738         # klasifikasi dengan SVM linear
739         clf = SVC(kernel="linear", random_state=42)
740
741         # klasifikasi dengan SVM linear
742         clf = SVC(kernel="linear", random_state=42)
743
744         # klasifikasi dengan SVM linear
745         clf = SVC(kernel="linear", random_state=42)
746
747         # klasifikasi dengan SVM linear
748         clf = SVC(kernel="linear", random_state=42)
749
750         # klasifikasi dengan SVM linear
751         clf = SVC(kernel="linear", random_state=42)
752
753         # klasifikasi dengan SVM linear
754         clf = SVC(kernel="linear", random_state=42)
755
756         # klasifikasi dengan SVM linear
757         clf = SVC(kernel="linear", random_state=42)
758
759         # klasifikasi dengan SVM linear
760         clf = SVC(kernel="linear", random_state=42)
761
762         # klasifikasi dengan SVM linear
763         clf = SVC(kernel="linear", random_state=42)
764
765         # klasifikasi dengan SVM linear
766         clf = SVC(kernel="linear", random_state=42)
767
768         # klasifikasi dengan SVM linear
769         clf = SVC(kernel="linear", random_state=42)
770
771         # klasifikasi dengan SVM linear
772         clf = SVC(kernel="linear", random_state=42)
773
774         # klasifikasi dengan SVM linear
775         clf = SVC(kernel="linear", random_state=42)
776
777         # klasifikasi dengan SVM linear
778         clf = SVC(kernel="linear", random_state=42)
779
780         # klasifikasi dengan SVM linear
781         clf = SVC(kernel="linear", random_state=42)
782
783         # klasifikasi dengan SVM linear
784         clf = SVC(kernel="linear", random_state=42)
785
786         # klasifikasi dengan SVM linear
787         clf = SVC(kernel="linear", random_state=42)
788
789         # klasifikasi dengan SVM linear
790         clf = SVC(kernel="linear", random_state=42)
791
792         # klasifikasi dengan SVM linear
793         clf = SVC(kernel="linear", random_state=42)
794
795         # klasifikasi dengan SVM linear
796         clf = SVC(kernel="linear", random_state=42)
797
798         # klasifikasi dengan SVM linear
799         clf = SVC(kernel="linear", random_state=42)
800
801         # klasifikasi dengan SVM linear
802         clf = SVC(kernel="linear", random_state=42)
803
804         # klasifikasi dengan SVM linear
805         clf = SVC(kernel="linear", random_state=42)
806
807         # klasifikasi dengan SVM linear
808         clf = SVC(kernel="linear", random_state=42)
809
810         # klasifikasi dengan SVM linear
811         clf = SVC(kernel="linear", random_state=42)
812
813         # klasifikasi dengan SVM linear
814         clf = SVC(kernel="linear", random_state=42)
815
816         # klasifikasi dengan SVM linear
817         clf = SVC(kernel="linear", random_state=42)
818
819         # klasifikasi dengan SVM linear
820         clf = SVC(kernel="linear", random_state=42)
821
822         # klasifikasi dengan SVM linear
823         clf = SVC(kernel="linear", random_state=42)
824
825         # klasifikasi dengan SVM linear
826         clf = SVC(kernel="linear", random_state=42)
827
828         # klasifikasi dengan SVM linear
829         clf = SVC(kernel="linear", random_state=42)
830
831         # klasifikasi dengan SVM linear
832         clf = SVC(kernel="linear", random_state=42)
833
834         # klasifikasi dengan SVM linear
835         clf = SVC(kernel="linear", random_state=42)
836
837         # klasifikasi dengan SVM linear
838         clf = SVC(kernel="linear", random_state=42)
839
840         # klasifikasi dengan SVM linear
841         clf = SVC(kernel="linear", random_state=42)
842
843         # klasifikasi dengan SVM linear
844         clf = SVC(kernel="linear", random_state=42)
845
846         # klasifikasi dengan SVM linear
847         clf = SVC(kernel="linear", random_state=42)
848
849         # klasifikasi dengan SVM linear
850         clf = SVC(kernel="linear", random_state=42)
851
852         # klasifikasi dengan SVM linear
853         clf = SVC(kernel="linear", random_state=42)
854
855         # klasifikasi dengan SVM linear
856         clf = SVC(kernel="linear", random_state=42)
857
858         # klasifikasi dengan SVM linear
859         clf = SVC(kernel="linear", random_state=42)
860
861         # klasifikasi dengan SVM linear
862         clf = SVC(kernel="linear", random_state=42)
863
864         # klasifikasi dengan SVM linear
865         clf = SVC(kernel="linear", random_state=42)
866
867         # klasifikasi dengan SVM linear
868         clf = SVC(kernel="linear", random_state=42)
869
870         # klasifikasi dengan SVM linear
871         clf = SVC(kernel="linear", random_state=42)
872
873         # klasifikasi dengan SVM linear
874         clf = SVC(kernel="linear", random_state=42)
875
876         # klasifikasi dengan SVM linear
877         clf = SVC(kernel="linear", random_state=42)
878
879         # klasifikasi dengan SVM linear
880         clf = SVC(kernel="linear", random_state=42)
881
882         # klasifikasi dengan SVM linear
883         clf = SVC(kernel="linear", random_state=42)
884
885         # klasifikasi dengan SVM linear
886         clf = SVC(kernel="linear", random_state=42)
887
888         # klasifikasi dengan SVM linear
889         clf = SVC(kernel="linear", random_state=42)
890
891         # klasifikasi dengan SVM linear
892         clf = SVC(kernel="linear", random_state=42)
893
894         # klasifikasi dengan SVM linear
895         clf = SVC(kernel="linear", random_state=42)
896
897         # klasifikasi dengan SVM linear
898         clf = SVC(kernel="linear", random_state=42)
899
900         # klasifikasi dengan SVM linear
901         clf = SVC(kernel="linear", random_state=42)
902
903         # klasifikasi dengan SVM linear
904         clf = SVC(kernel="linear", random_state=42)
905
906         # klasifikasi dengan SVM linear
907         clf = SVC(kernel="linear", random_state=42)
908
909         # klasifikasi dengan SVM linear
910         clf = SVC(kernel="linear", random_state=42)
911
912         # klasifikasi dengan SVM linear
913         clf = SVC(kernel="linear", random_state=42)
914
915         # klasifikasi dengan SVM linear
916         clf = SVC(kernel="linear", random_state=42)
917
918         # klasifikasi dengan SVM linear
919         clf = SVC(kernel="linear", random_state=42)
920
921         # klasifikasi dengan SVM linear
922         clf = SVC(kernel="linear", random_state=42)
923
924         # klasifikasi dengan SVM linear
925         clf = SVC(kernel="linear", random_state=42)
926
927         # klasifikasi dengan SVM linear
928         clf = SVC(kernel="linear", random_state=42)
929
930         # klasifikasi dengan SVM linear
931         clf = SVC(kernel="linear", random_state=42)
932
933         # klasifikasi dengan SVM linear
934         clf = SVC(kernel="linear", random_state=42)
935
936         # klasifikasi dengan SVM linear
937         clf = SVC(kernel="linear", random_state=42)
938
939         # klasifikasi dengan SVM linear
940         clf = SVC(kernel="linear", random_state=42)
941
942         # klasifikasi dengan SVM linear
943         clf = SVC(kernel="linear", random_state=42)
944
945         # klasifikasi dengan SVM linear
946         clf = SVC(kernel="linear", random_state=42)
947
948         # klasifikasi dengan SVM linear
949         clf = SVC(kernel="linear", random_state=42)
950
951         # klasifikasi dengan SVM linear
952         clf = SVC(kernel="linear", random_state=42)
953
954         # klasifikasi dengan SVM linear
955         clf = SVC(kernel="linear", random_state=42)
956
957         # klasifikasi dengan SVM linear
958         clf = SVC(kernel="linear", random_state=42)
959
960         # klasifikasi dengan SVM linear
961         clf = SVC(kernel="linear", random_state=42)
962
963         # klasifikasi dengan SVM linear
964         clf = SVC(kernel="linear", random_state=42)
965
966         # klasifikasi dengan SVM linear
967         clf = SVC(kernel="linear", random_state=42)
968
969         # klasifikasi dengan SVM linear
970         clf = SVC(kernel="linear", random_state=42)
971
972         # klasifikasi dengan SVM linear
973         clf = SVC(kernel="linear", random_state=42)
974
975         # klasifikasi dengan SVM linear
976         clf = SVC(kernel="linear", random_state=42)
977
978         # klasifikasi dengan SVM linear
979         clf = SVC(kernel="linear", random_state=42)
980
981         # klasifikasi dengan SVM linear
982         clf = SVC(kernel="linear", random_state=42)
983
984         # klasifikasi dengan SVM linear
985         clf = SVC(kernel="linear", random_state=42)
986
987         # klasifikasi dengan SVM linear
988         clf = SVC(kernel="linear", random_state=42)
989
990         # klasifikasi dengan SVM linear
991         clf = SVC(kernel="linear", random_state=42)
992
993         # klasifikasi dengan SVM linear
994         clf = SVC(kernel="linear", random_state=42)
995
996         # klasifikasi dengan SVM linear
997         clf = SVC(kernel="linear", random_state=42)
998
999         # klasifikasi dengan SVM linear
1000        
```

2. Screenshot Output

```

=====
EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSTRAKSI DAN MATCHING METODE
=====

[METODE] Mengevaluasi Feature Extractor: SIFT
-> Rata-rata Keypoints: 250.80
-> Rata-rata Waktu Ekstraksi: 26.586 ms
-> Dimensi Deskriptor: 128
-> Rata-rata Geometri Inliers (RANSAC): 123.50

[METODE] Mengevaluasi Feature Extractor: ORB
-> Rata-rata Keypoints: 653.40
-> Rata-rata Waktu Ekstraksi: 19.768 ms
-> Dimensi Deskriptor: 32
-> Rata-rata Geometri Inliers (RANSAC): 315.30

=====
EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BoVW) & SVM
=====

[BoVW] Akurasi SVM dengan K=10 : 50.0%
[BoVW] Akurasi SVM dengan K=20 : 60.0%

[Confusion Matrix untuk BoVW K=20]
[[1 0 1 0]
 [0 0 1 1]
 [0 1 0 1]
 [0 0 2 0]]

Classification Report:
              precision    recall  f1-score   support

      buku             1.00      0.50      0.67         2
       mug             0.00      0.00      0.00         2
      botol            1.00      0.50      0.67         2
      mainan          0.50      1.00      0.67         2
      remote          0.50      1.00      0.67         2

      accuracy                   0.60         10
    macro avg                   0.60      0.53         10
   weighted avg                   0.60      0.53         10

[BoVW] Akurasi SVM dengan K=50 : 70.0%
[BoVW] Akurasi SVM dengan K=100: 50.0%

=====
EKSPLORASI TUGAS 4: REDUKSI DIMENSI DESKRIPTOR SIFT DENGAN PCA
=====

[PCA] Komponen: 16 | Total Variance Retained: 65.27%
[PCA] Komponen: 32 | Total Variance Retained: 81.72%
[PCA] Komponen: 64 | Total Variance Retained: 93.99%
[PCA] Komponen: 128 | Total Variance Retained: 100.00%

```

File lengkap ada di repo github (Pertemuan 11 > dataset)

<https://github.com/NirxTech/Pengolahan-Citra-Digital>